

ProjectMatch

AI-Powered Constraint-Aware Team Formation Platform — Development Timeline Documentation

1. Project Executive Overview

ProjectMatch is a sophisticated multi-agent AI framework engineered to assemble optimal, constraint-aware project teams. Unlike naive skill matchers that rely solely on surface-level keyword similarity, ProjectMatch implements a 3-Layer Prompt-Reasoning Pipeline powered by **gemini-3.6-flash**. The pipeline extracts structured JSON from unstructured candidate self-reports, runs Chain-of-Thought (CoT) matching against project parameters, and subjects team compositions to an automated Layer 3 critique loop.

Project Name	ProjectMatch (FAST SRM)
Core Architecture	3-Layer Prompt-Reasoning Pipeline (Extraction → Selection → Validation)
Primary AI Model	Google Gemini (gemini-3.6-flash) with Deterministic Fallbacks
Backend Stack	FastAPI (Python), Motor (Async MongoDB), PyMongo
Frontend Stack	React, Vite, TailwindCSS, Lucide Icons, Recharts
Document Date	August 25, 2026

2. Comprehensive Development Timeline & Phase Roadmap

The ProjectMatch platform was developed through structured phases, moving from architecture formulation to full-stack implementation, critique feedback loops, and evaluation benchmarking.

Phase / Milestone	Key Accomplishments & Deliverables	Core Stack / Modules	Status
Phase 1: Architecture & PRD Definition	- Formulated 3-layer reasoning concepts - Drafted ProjectMatch PRD specifications - Designed multi-agent feedback loop strategy	ProjectMatch_PRD.md System Architecture	Completed
Phase 2: Database Schema & Seeding	- Configured MongoDB models for Candidates, Projects, & Matches - Developed automated dataset seeder (<code>seed_data.py</code>) - Established async database connection via Motor	FastAPI, MongoDB Motor, Pydantic	Completed
Phase 3: Layered AI Pipeline	Layer 1: Profile extraction & skill confidence calibration Layer 2: Constraint-aware CoT candidate selection Layer 3: Self-critique & auto-revision cycle (max 2) Fallback: Regex & heuristic matching engine	Gemini 3.6 Flash <code>pipeline.py`</code> Prompts Modules	Completed

Phase 4: FastAPI REST Layer	• Implemented candidate CRUD routes • Built project requirements management routes • Designed matching execution & live analytics APIs	FastAPI Uvicorn, CORS	Completed
Phase 5: React Dashboard Frontend	• SPA shell with tabbed navigation • Interactive Candidate & Project management views • Real-time matching pipeline viewer & CoT display • Analytics view comparing baseline vs multi-agent	React, Vite TailwindCSS Lucide Icons	Completed
Phase 6: Evaluation & Benchmarking	• Integrated timing benchmarks per pipeline layer • Built Naive Baseline prompt comparator • Added multi-agent revision telemetry tracking	`naive_baseline.py` Analytics Dashboard	Completed
Phase 7: Cloud Deployment Readiness	• Configured powershell setup scripts (`start_all.ps1`) • Prepared MongoDB Atlas & Cloud hosting workflow • Generated cloud deployment & PDF documentation	Render / Vercel MongoDB Atlas ReportLab	Completed

3. Technical Architecture & Key Pipeline Features

- **Layer 1 (Profile Normalization):** Takes unstructured self-descriptions and parses them into standardized skill sets with explicit confidence levels (low/medium/high) and flag uncertain fields.
- **Layer 2 (Constraint-Aware Matching):** Matches normalized candidates against target project rules (tech stack, role split, availability), generating an explicit Chain-of-Thought (CoT) selection summary.
- **Layer 3 (Critique & Revision Loop):** Evaluates proposed teams for critical risks (e.g. single point of failure, role imbalance, overload). Instructs Layer 2 to revise up to 2 times if flaws are detected.
- **Deterministic Fallback Mechanism:** Guarantees system reliability. If API rate limits or network issues occur, the pipeline gracefully seamlessly falls back to hardcoded regex and heuristic matching.

4. Summary & Cloud Deployment Status

The ProjectMatch system is fully implemented, verified, and ready for production deployment. The backend can be deployed seamlessly to Render/Cloud Run using MongoDB Atlas, while the frontend can be hosted on static cloud edge networks such as Vercel or Netlify.